Universidad de los Andes
Departamento de Ingeniería de Sistemas y Computación
ISIS 1311 – Tecnología e Infraestructura de Cómputo

Caso 3 - 20261

Concurrencia y Sincronización de Procesos

El propósito de este caso es aplicar los conceptos de concurrencia y sincronización de procesos vistos en clase, a través del diseño e implementación de un simulador para un sistema gestor de recursos para máquinas virtuales.

1. Objetivos:

- Modelar interacciones concurrentes entre múltiples threads que compiten por recursos compartidos.
- Implementar mecanismos de sincronización utilizando exclusivamente las primitivas básicas de Java (synchronized, wait, notify, notifyAll, yield, join, CyclicBarrier).
- Usar diferentes patrones de espera: espera pasiva y espera semi-activa.
- Analizar situaciones típicas de concurrencia, como condiciones de carrera, bloqueos mutuos y pérdidas de solicitudes, proponiendo soluciones correctas mediante sincronización.

MUY IMPORTANTE: El entregable de este caso debe ser **100% de su autoría**.

A diferencia de los casos previos, donde usar AI no se recomendaba pero se permitía bajo ciertas condiciones, para este caso -**no está permitido-** usar ayudas (incluyendo chatbots o tecnologías similares). El incumplimiento de lo anterior resultará en una calificación de cero (0.0) para el caso.

Tenga en cuenta que los casos están diseñados para apoyar su proceso de aprendizaje e identificar y resolver sus dudas sobre los temas estudiados antes del parcial. Además, para comprender el tema de este módulo es muy importante que el grupo desarrolle el caso a conciencia, identifique los retos de sincronización que se presentan y los resuelva de forma apropiada.

Finalmente, recuerde que en el parcial habrá una pregunta sobre el caso.

2. El Caso: Sistema Gestor de Recursos para Máquinas Virtuales

Queremos simular un sistema que gestiona los recursos disponibles para la creación de máquinas virtuales. El sistema debe coordinar la recepción de solicitudes, asignación y liberación de recursos y creación de máquinas virtuales.

A. Arquitectura general del sistema

La Figura 1 ilustra cómo está organizado el sistema.

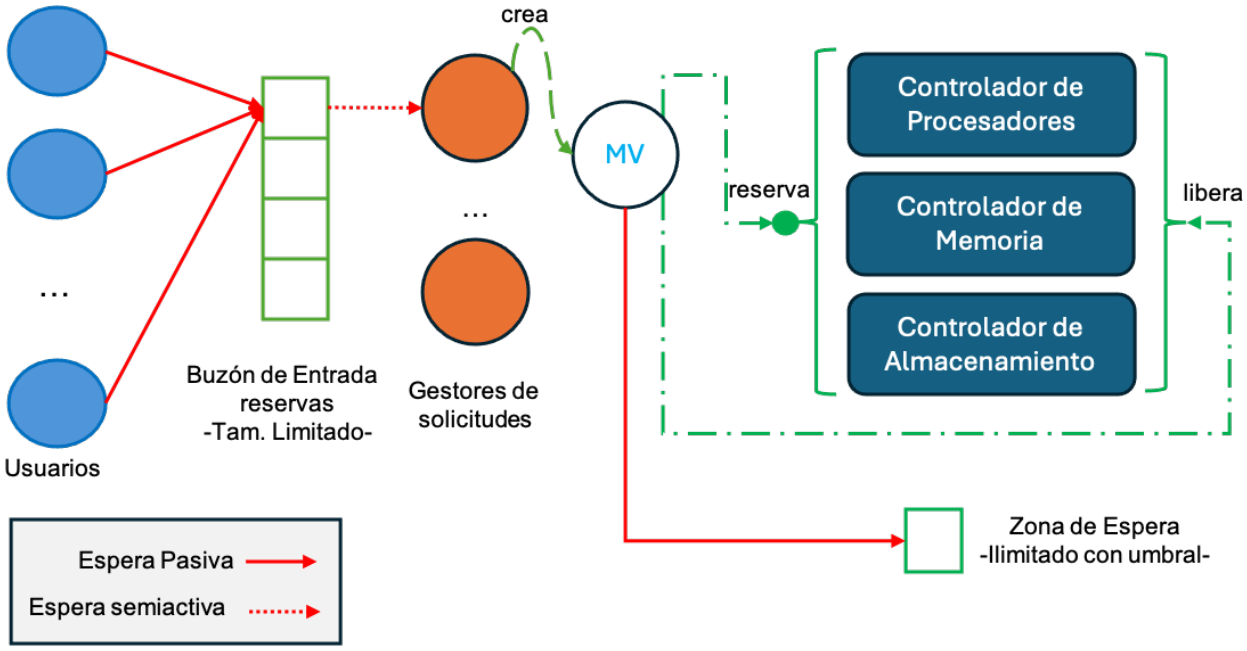


Figura 1. Arquitectura del Sistema Gestor de Recursos para MVs

Descripción de los actores:

1. Usuario
    - Son los responsables de generar las solicitudes de creación de MVs.
    - Cada usuario produce un número asignado de solicitudes.
    - Cada usuario deposita su solicitud en el buzón de entrada.
    - Cada usuario termina cuando ha generado todas las solicitudes.
2. Gestor de solicitudes
    - Son los responsables de procesar las solicitudes una por una. Dando prioridad a las solicitudes pendientes.
    - Por cada solicitud en el buzón de entrada crea una máquina virtual (MV).
    - Cada gestor termina cuando se han procesado todas las solicitudes.

3. MV
    - Intenta reservar los recursos en orden: procesadores, memoria y almacenamiento.
    - Si no puede completar la solicitud se duerme sobre la zona de espera.
    - Si completa la solicitud, se ejecuta durante un tiempo aleatorio, al final libera los recursos y termina.
    - Al liberar los recursos debe despertar otras VMs que estén en espera. Usted debe determinar si todas o solo una.
4. Objetos de control
    - Son tres objetos independientes (no son threads) que sirven para controlar los recursos asignados y libres:
        1. El objeto controlador de procesadores debe registrar cuáles procesadores están asignados y cuáles están libres.
        2. El objeto controlador de memoria debe registrar cuáles páginas están asignadas y cuáles están libres. Adicionalmente, deberá traducir cada solicitud que reciba (en GiB) al múltiplo más cercano del tamaño de una página.
        3. El objeto controlador de almacenamiento debe registrar cuáles bloques están asignados y cuáles están libres. Deberá traducir cada solicitud que reciba (en GiB) al múltiplo más cercano del tamaño de bloque.
    - Observe que en cada caso el objeto controlador no solo sabe “cuánto recurso” hay disponible, sino cuáles partes.
    - Adicionalmente, cada objeto controlador debe tener el registro de la máquina virtual que tiene el recurso.

B. Funcionamiento del Sistema

Condiciones de funcionamiento del sistema:

1. Usuarios

- Cada Usuario (thread productor) se inicia con un número fijo de solicitudes que debe generar. Este número se calcula con un valor base en el archivo de configuración multiplicado por el número de identificación del usuario (usaremos identificadores secuenciales, empezando en 1).
- Cada usuario genera sus solicitudes con un identificador único. El identificador debe generarse con el identificador del usuario y un secuencial.
- Cada solicitud debe incluir la descripción de los recursos solicitados: número de procesadores, memoria en GiB y espacio en disco en GiB. Los valores serán generados de forma seudoaleatoria así:

    - #procesadores: entre 1 y el entero más cercano a 3/4 de la capacidad máxima del recurso.
    - Memoria: entre 1 y el entero más cercano a 3/4 de la capacidad máxima del recurso. El valor debe redondearse a la potencia de dos más alta que sea inferior al valor generado.
    - Almacenamiento: entre 1 y el entero más cercano a 3/4 de la capacidad máximo del recurso. El valor debe redondearse al múltiplo de dos más alto que sea inferior al valor generado.
- Los usuarios depositan sus solicitudes en un buzón con capacidad limitada. Si el buzón está lleno deben esperar (espera pasiva).
- Cada usuario termina cuando termina de producir el número asignado de solicitudes. Al terminar reporta su terminación sobre el buzón de entrada (de esta forma los gestores pueden consultar el número de usuarios activos en el sistema).

2. Gestor de solicitudes:

- Si no hay solicitudes, un gestor debe esperar (espera semi-actia)
- Procesa las solicitudes una por una. Primero revisa el número de MVs pendientes en la zona de espera:
    - La zona de espera tendrá un umbral establecido.
    - Si el número de MVs en espera es inferior al umbral, el gestor procesa una solicitud del buzón de entrada.
    - Si el número de MVs en espera es igual o superior al umbral, el gestor debe esperar a que <u>todas</u> las solicitudes pendientes se procesen (no permite el ingreso de solicitudes adicionales hasta terminar con las pendientes). ¿Qué problema resuelve esta manera de procesar solicitudes?
- Por cada solicitud que sí puede procesar crea una MV (un thread), con la configuración correspondiente e inicia su ejecución.
- Cada gestor termina cuando todos los usuarios han terminado y el buzón de entrada está vacío.

3. MV

- Una máquina reserva los recursos que necesita -uno a uno- y siempre en el orden establecido: procesador, memoria y disco. ¿Por qué es importante que todas sigan el mismo orden de solicitud?
- Cuando reserva un recurso, este se asigna únicamente si en ese momento existe una cantidad suficiente disponible.
- Si una máquina logra reservar un recurso, este queda asignado exclusivamente para ella mientras intenta completar su configuración.

- Si, después de haber recibido uno o más recursos, encuentra que el siguiente recurso que necesita no está disponible, debe liberar inmediatamente todos los recursos que había reservado previamente. Por ejemplo, si una máquina ya recibió procesadores y memoria, pero no puede obtener el disco requerido, deberá devolver tanto los procesadores como la memoria. ¿por qué es importante liberar los recursos asignados cuando no se logra completar la reserva?
- Si no pudo reservar todos los recursos necesarios: libera los recursos reservados y la máquina debe entrar en una zona de espera (espera pasiva). Cuando cambie la disponibilidad de recursos, podrá volver a intentar obtener su configuración completa, comenzando nuevamente por los procesadores.
- Una máquina solamente puede comenzar su ejecución cuando haya conseguido exitosamente los tres tipos de recursos requeridos.
- Una vez configurada, la máquina utiliza los recursos durante un tiempo aleatorio entre 4000 y 40000 ms.
- Cuando termine su ejecución, la máquina deberá liberar todos los recursos que tenía asignados y despertar otra máquina para que use los recursos. ¿debería despertar una sola o todas? ¿por qué?

C. Terminación del sistema

- Los usuarios terminan cuando han generado todas las solicitudes asignadas.
- Los gestores terminan cuando todos los usuarios han terminado y el buzón de entrada está vacío.
- Todas las máquinas virtuales deben terminar su ejecución.
- El buzón debe quedar vacío.
- La zona de espera debe quedar vacía.
- Los recursos libres en los controladores deben volver al estado inicial (todo el recurso libre).

D. Parámetros configurables

El programa recibe un archivo texto que define los siguientes parámetros:

- Número de usuarios (nu).
- Número base de eventos (base).
- Número de gestores (ng).
- Capacidad del buzón del buzón de entrada (tam1).
- Capacidad de la zona de espera (tam2).
- Número de procesadores en el sistema (procesadores).
- Capacidad en memoria y tamaño de página (memoria -en GiB-, página en KiB).
- Capacidad en almacenamiento y tamaño de bloque. (almacenamiento -en GiB-, bloque en KiB)

3. Entrega

- En un archivo .zip entregue el código fuente del programa, y un informe explicando el diseño (diagrama de clase) y funcionamiento del programa.
- En el informe explique, para cada pareja de objetos que interactúan, cómo se realiza la sincronización, así como el funcionamiento global del sistema.
- Incluya las respuestas a las preguntas planteadas a lo largo del enunciado (justificando cada respuesta en un párrafo de máximo 10 líneas).
- Explique cómo implementó la espera "total" del gestor en el caso donde las solicitudes pendientes son >= umbral.
- Incluya una sección de validación:
    - Mencione qué aspectos verificó
    - Por cada aspecto describa qué pruebas hizo para verificar el funcionamiento correcto de ese aspecto
- El nombre del archivo zip debe ser: caso3_login1_login2.zip

4. Condiciones de entrega

- El trabajo se debe realizar en grupos de 2 estudiantes. No debe haber consultas entre grupos. Recuerde que debe conformar un grupo diferente a los casos anteriores y debe trabajar en grupo.
- El grupo responde solidariamente por el contenido de todo el trabajo, y lo elabora conjuntamente (no es trabajo en grupo repartirse puntos o trabajos diferentes).
- Tenga en cuenta las reglas establecidas en el programa del curso sobre la obligatoriedad del trabajo en grupo. Si no cumple con las reglas habrá una penalización de 0,5/5.
- En el parcial se incluirá una pregunta sobre el caso.
- El proyecto debe ser entregado por Bloque Neón por uno solo de los integrantes del grupo. **Al comienzo del informe, deben estar los nombres y carnés de los integrantes del grupo.** Si un integrante no aparece en el documento entregado, el grupo podrá informarlo posteriormente, sin embargo, habrá una penalización: la calificación asignada será distribuida (dividida de forma equitativa) entre los integrantes del grupo.
- **Se debe entregar por Bloque Neón a más tardar el 4 de noviembre a las 11:59 p.m.**
- **La fecha de entrega no será movida.**
- **Política de entrega tarde.** Para las entregas tarde, se aplicará la siguiente política: por cada 30 minutos o fracción de retraso, con respecto a la hora de entrega establecida en Bloque Neón, habrá una penalización de 0,5/5.
- En Bloque Neón, **es necesario inscribirse** en alguno de los grupos disponibles para poder acceder a la actividad de entrega del caso. Para ello, debe ingresar a la sección Grupos → Ver grupos disponibles → Caso 3. Es muy importante asegurarse de que ambos integrantes estén registrados en el mismo grupo; de lo contrario, quien no se inscriba no recibirá la información de la calificación.

5. Cronograma Propuesto

- Oct. 20-22: Lectura y comprensión del enunciado, definición de grupos
- Oct. 23-26: Diseño
- Oct. 26-Oct. 30: Implementación y pruebas
- Oct. 30-Nov. 3: Redacción del informe
- Nov. 4: Entrega